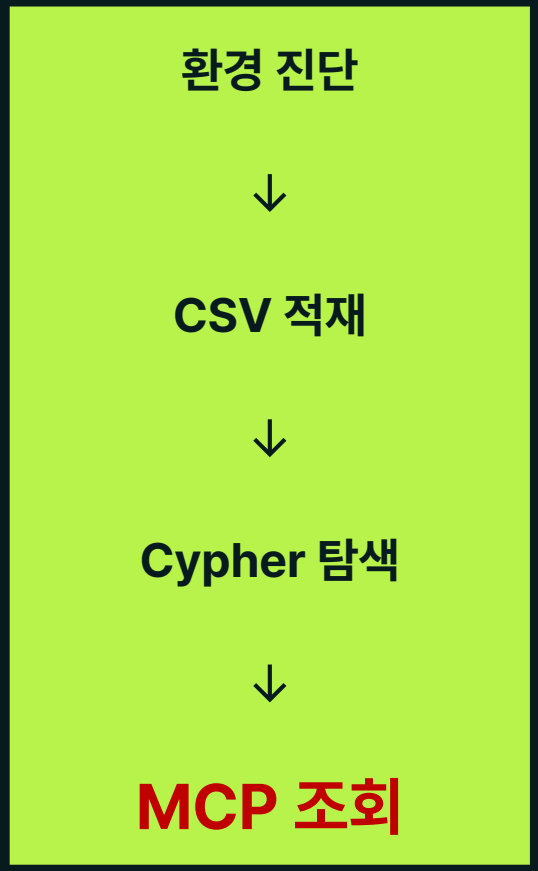
한국 자동차 산업 지식그래프 구축 실습

Claude Code로 Neo4j 그래프를 만들고 MCP로 검증합니다



K-Auto KG Stage 1 · Obsidian 종합실습

실습은 다섯 구간을 차례로 통과합니다

P0~P14 프롬프트를 순서대로 입력하고, 구간마다 성공을 로그로 판정한다

01	기본 개념과 준비	**구성요소 · 그래프 DB · MCP** Claude Code · Neo4j · Cypher · 관계 8종
02	환경 진단과 DB 시작	**P0 ~ P3** 현황 조사 · 설치 계획 · .env · Docker
03	데이터와 온톨로지 적재	**P4 ~ P7** CSV 테스트 · 스키마 · MERGE · 검증
04	그래프 탐색과 관계 분석	**P8 ~ P10** HTML 미리보기 · Cypher · 출처 추적
05	MCP 연결 · 개선 · 평가	**P11 ~ P14** read-only MCP · 진단 · 제출 · 2단계

이번 실습은 완성된 그래프를 구경하지 않고 직접 재현합니다

교육용 시드 데이터 기준 — 수치는 산업 전체가 아니라 제공 패키지의 기준값이다

30	7	24	42	8
기업	밸류체인	핵심역량	기업 간 관계	관계 유형



출력보다 입력한 명령 · 확인 절차 · 출처 해석을 평가합니다

실습을 마치면 지시 · 이해 · 구별 세 가지를 할 수 있습니다

학습성과 6개를 세 묶음으로 정리했다

지시한다

Claude Code 운용

- 단계별로 지시
- 오류 재검증 요구

이해한다

그래프 DB 구조

- 노드·관계·속성
- 세 화면 역할 구분

구별한다

관계의 증거 수준

- 계약과 납품 구분
- 출처까지 추적

최종 제출: 프롬프트 3개+ · 판정 로그 · 화면 2개 · 관계 1건 분석 · 수정 Cypher 1개

※ 비밀번호·토큰·사내 비공개 자료는 제출하지 않는다

‘완료했습니다’ 한 문장이 아니라 로그로 증명합니다

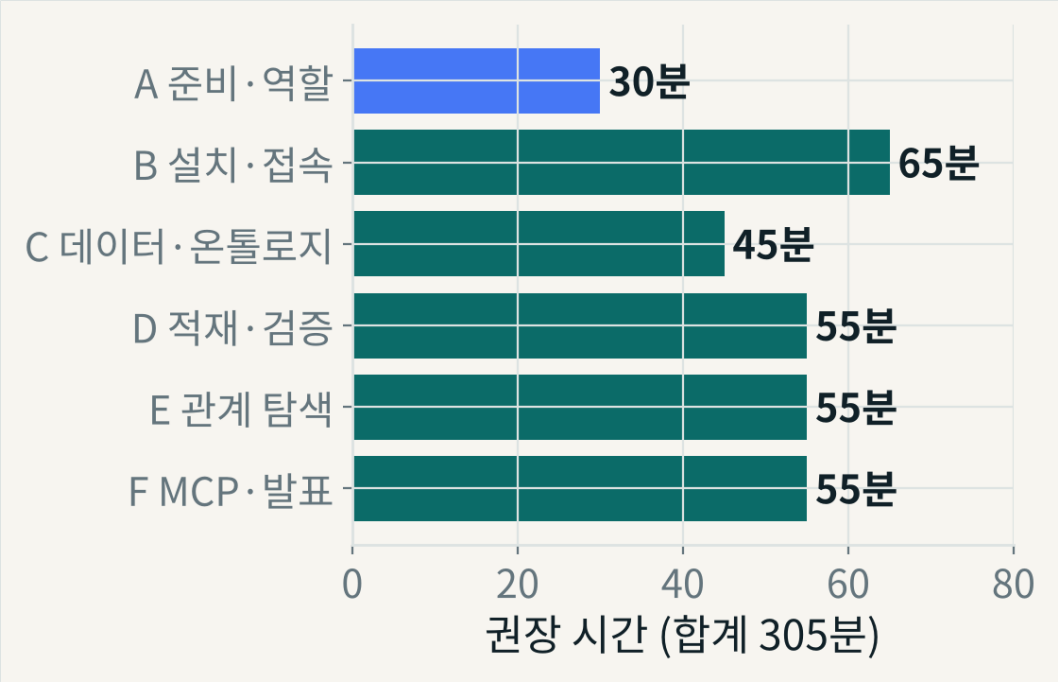
설치·접속 구간이 가장 오래 걸립니다

강의 운영표 6구간 — 합계 약 5시간, Docker 첫 설치는 별도 시간이 든다

운영 원칙

Plan A / B

- 설치돼 있으면 생략
- 연결만 검증
- 막히면 Plan B



시간표는 가이드이며, 막힌 수강생은 Plan B로 먼저 성공시킵니다

01

기본 개념과 준비

무엇이 무엇을 하는가 — 구성요소 · 그래프 DB · Cypher · MCP · 준비물

일곱 가지 구성요소가 각자 한 가지 일을 맡습니다

이름보다 '수강생이 실제로 하는 행동'을 기억한다

개발 에이전트

Claude Code

프롬프트로 지시

컨테이너 실행

Docker

Neo4j 격리 실행

그래프 DB

Neo4j

localhost:7474

질의 언어

Cypher

MATCH·RETURN

도구 연결 표준

MCP

claude mcp list · /mcp

코드 연결

Python Driver

CSV 적재·검증 스크립트

미리보기

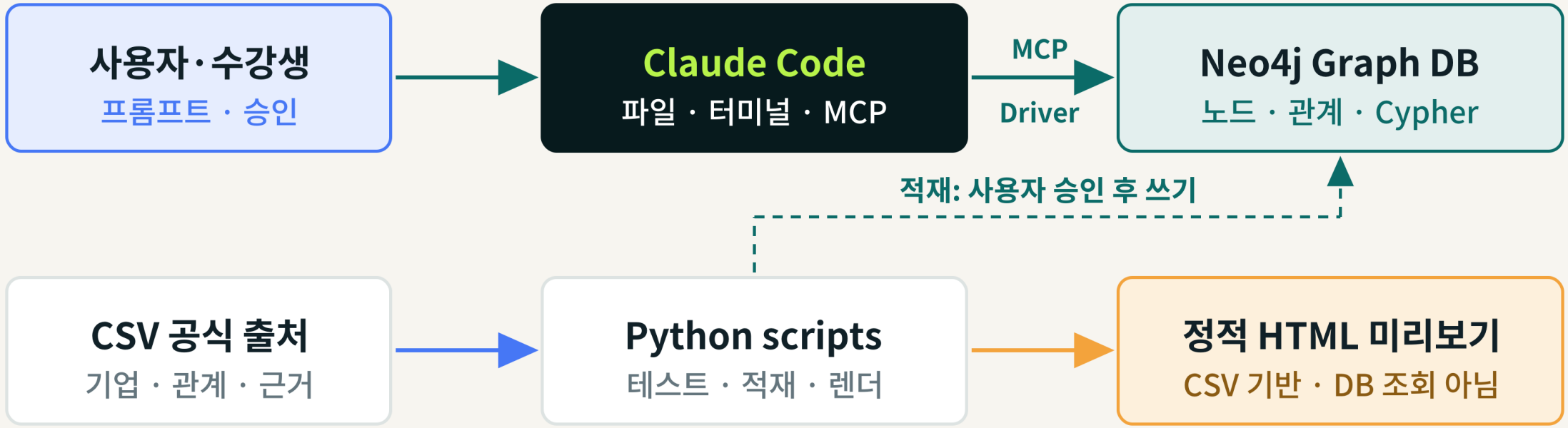
정적 HTML

DB와 실시간 연결 없음

Claude Code는 손, Neo4j는 저장소, MCP는 Claude가 저장소를 읽는 통로입니다

데이터가 흐르는 길은 두 갈래입니다

위: 사용자 → Claude Code → Neo4j (실제 DB) · 아래: CSV → Python → HTML (미리보기)



HTML을 열었다고 Neo4j에 데이터가 적재된 것은 아닙니다

그래프 DB는 '대상'과 '연결'을 그대로 저장합니다

Graph Database — 표의 행 대신 노드, 조인 대신 관계로 데이터를 담는다

왜 그래프인가

관계가 곧 데이터

- 연결을 바로 조회
- 다단계 경로 탐색
- 관계마다 근거 저장



노드(Node)	기업·역량·시스템 같은 '대상' = 원
관계(Relationship)	방향과 타입이 있는 '연결' = 화살표
속성(Property)	id · name_ko · evidence_url 같은 값

산업지도는 '누가 누구와 어떤 관계인가'가 핵심이라 그래프가 맞습니다

Cypher는 그림 그리듯 패턴을 적어 질문합니다

괄호 ( )는 노드, 화살표 -[ ]→는 관계 — Neo4j Browser 상단 입력창에서 실행한다

예시 · 현대모비스가 가진 역량

```
MATCH (c:KAutoCompany {id:'company.mobis'})
      -[:HAS_CAPABILITY]->
      (p:KAutoCapability)
RETURN c.name_ko AS 기업,
       p.name_ko AS 역량;
```

읽기 (안전)

MATCH 찾기 · WHERE 조건
RETURN 결과 돌려받기

쓰기 (승인 필요)

CREATE · MERGE · DELETE
데이터를 바꾼다

이번 실습

탐색은 읽기 전용만 쓴다

SQL과 비슷해 보여도 Cypher이며, 쓰기 명령은 승인 후에만 실행합니다

적재는 Python Driver가, 조회는 MCP가 맡습니다

MCP(Model Context Protocol) — Claude Code가 외부 도구를 호출하게 하는 연결 표준

PYTHON DRIVER

터미널에서 쓰기

- import_neo4j.py로 적재
- verify_neo4j.py로 검증
- 쓰기 전 사용자 승인

데이터를 넣는 길



NEO4J MCP

Claude가 읽기

- mcp_launcher.py로 등록
- read-only로 연결
- 비밀번호는 .env에서만

등록 ≠ 실제 호출 성공

MCP가 실패했는데 Python으로 대신 조회하고 'MCP 성공'이라 적지 않습니다

기업 간 선 하나하나는 서로 다른 주장입니다

기업↔기업 관계 8종과 시드 건수 — 오른쪽 빨간 글씨가 해석할 때 가장 조심할 점이다

12	동일 그룹	AFFILIATED_WITH	지분 보유 아님	12	컨소시엄 참여	CO_MEMBER_OF_CONSORTIUM	직접 거래 아님
6	공동 연구개발	JOINT_RESEARCH_WITH	양산·판매 아님	4	기술 협력	TECH_PARTNERS_WITH	독점 아님
3	공급계약	SUPPLY_AGREEMENT_WITH	납품 완료 아님	3	JV 계획 승인	JOINT_VENTURE_PLAN_WITH	현재 운영 아님
1	기존 공급 보도	SUPPLIES_TO	현재 물량 미확인	1	배터리 협력	BATTERY_COOPERATION_WITH	공급계약 아님

그래프의 모든 선을 거래로 읽지 않습니다 — 납품·계약은 초록 두 칸뿐입니다

준비물은 각자 확인하고, 파괴적 명령은 강사가 막습니다

회사 PC라 Docker 설치가 막혀 있으면 강사에게 기존 Neo4j나 실습 DB를 요청한다

수강생 준비 체크

내 PC에서 확인

- kauto_kg_mvp 안에서 터미널
- claude · python · docker 버전
- 포트 7474 · 7687 사용 권한

버전은 고정하지 말고 점검

실습 금지 명령

한꺼번에 승인 금지

- Docker 볼륨 삭제
- MATCH (n) DETACH DELETE n
- 전역 MCP 비밀번호 저장

쓰기·삭제는 설명 후 승인

비밀번호는 공개 채팅 · 공유 문서 · Git 어디에도 적지 않습니다

Vault는 교재이고 프로젝트 폴더는 작업장입니다

ZIP을 풀고 Obsidian에서 Open folder as vault(폴더를 보관함으로 열기)를 선택한다

지킬 것

폴더 구조 유지

- 경로를 바꾸지 않는다
- 명령은 터미널에서
- .env는 ZIP에 없다

KAuto_Obsidian_Stage1_Complete/	Vault 최상위
00_시작_여기.md	학습 순서
KAuto_Stage1_종합실습교재.md	본편 P0~P14
assets/svg/	인포그래픽 11종
KAuto_Project_Source/kauto_kg_mvp/	터미널은 여기서
data/ scripts/ queries/ tests/	CSV·코드·Cypher
.env.example → .env	비밀번호 · 공유 금지
student_report_TEMPLATE.md	제출 보고서

data · scripts · queries 상대경로는 kauto_kg_mvp 폴더에서만 맞습니다

터미널은 반드시 kauto_kg_mvp 폴더에서 엽니다

현재 폴더가 곧 명령 실행 위치다 — 마지막 폴더명을 눈으로 확인한다

Windows · PowerShell

```
# 1) 탐색기에서 kauto_kg_mvp 열기
# 2) 주소창에 powershell 입력 → Enter
Get-Location
claude
```

macOS · Terminal

```
# 1) Terminal에서 cd + 공백 입력
# 2) 폴더를 끌어다 놓고 Enter
pwd
claude
```

claude 명령이 없으면 미설치 또는 PATH 문제이므로 P1에서 설치를 요청합니다

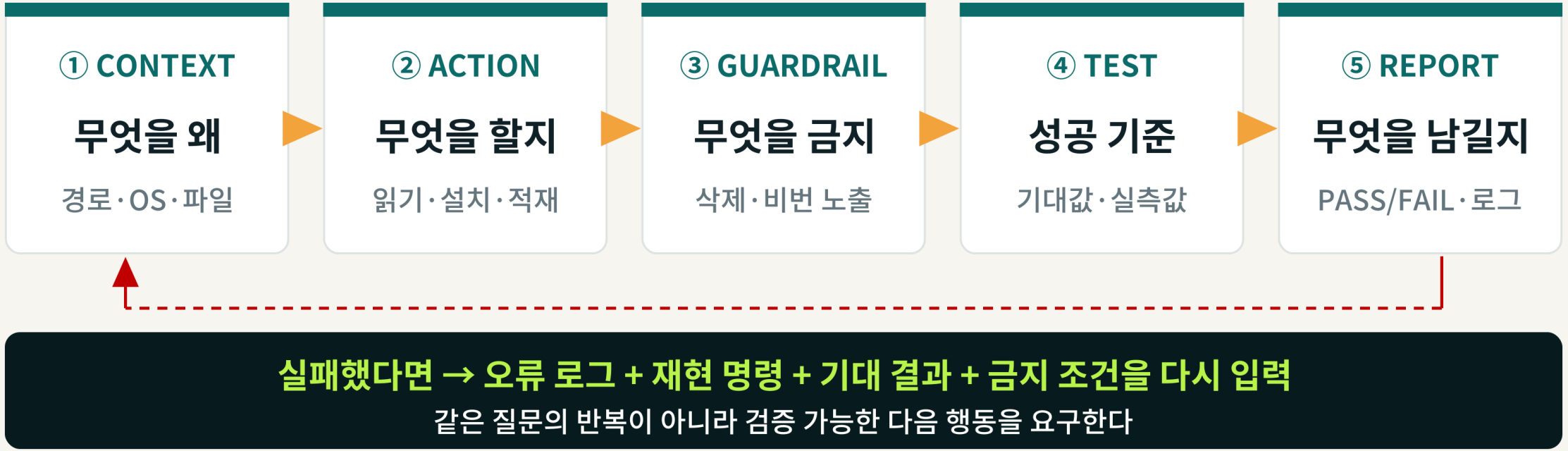
02

환경 진단과 Neo4j 시작

P0 ~ P3 — 설치하기 전에 먼저 진단하고, 승인 경계에서 멈춘다

모든 프롬프트는 다섯 칸을 채워서 보냅니다

상황 → 지시 → 금지 → 검증 → 보고 — Claude Code가 '완료'라고만 답하면 근거를 요구한다



P0~P14는 모두 이 다섯 칸을 채운 예시입니다

첫 프롬프트는 설치하지 않고 현황만 조사합니다

어디: kauto_kg_mvp 폴더에서 실행한 Claude Code 대화창

P0 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 | P11 | P12 | P13 | P14

P0 · Claude Code 채팅에 붙여 넣기

```
너는 초보자를 도와주는 환경구축 코치다.
현재 폴더의 README.md, CLAUDE.md,
CLAUDE_SETUP_PROMPT.md를 읽고 운영체제, 셸,
Python, Claude Code, Docker/Compose,
7474·7687 포트, 기존 Neo4j 및 MCP 등록
상태를 확인해줘. 지금은 설치하거나 설정을
바꾸지 말고, (1) 이미 준비된 것 (2) 미설치
(3) 확인 불가를 표로 보고해줘.
```

예상 출력

OS·셸별 준비/미설치/확인불가 표와 다음 행동 제안

확인

실제 탐지 결과인가?
근거 없는 '설치됨'은 없는가?

다음 행동은 '어디 → 입력 → 출력 → 확인' 순서로 하나씩 받습니다

설치 계획은 승인이 필요한 지점 직전에 멈추게 합니다

이미 있는 프로그램은 재설치하지 않고, 공식 설치 경로만 쓴다

P0 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 | P11 | P12 | P13 | P14

P1 · Claude Code 채팅에 붙여 넣기

```
P0 진단 결과를 기준으로 이미 있는 프로그램은
재설치하지 마. 필요할 때만 공식 설치 경로를
사용해 Docker Desktop, Python, Claude Code를
준비할 계획을 세워줘. 관리자 승인·로그인·
비밀번호 입력은 내게 안내하고 직접 승인이
필요한 작업 직전에 멈춰줘. 기존 DB와 도커
볼륨을 삭제하지 마. 설치 전후의 버전 명령으로
성공을 재확인해줘.
```

프롬프트의 금지 조건

재설치 금지
기존 DB·볼륨 삭제 금지

프롬프트의 검증 조건

설치 전후 버전 명령으로 재확인

관리자 승인 · 로그인 · 비밀번호 입력은 수강생이 직접 합니다

같은 진단을 터미널 명령으로 직접 확인할 수 있습니다

T1 — Claude Code 보고와 내 터미널 결과가 같은지 대조한다

T1 · Windows PowerShell

```
Get-Location
py --version
claude --version
docker --version
docker compose version
claude mcp list
```

T1 · macOS Terminal

```
pwd
python3 --version
claude --version
docker --version
docker compose version
claude mcp list
```

docker --version만 성공하고 compose ps가 실패하면 Desktop 엔진이 꺼져 있을 수 있습니다

비밀번호는 Claude가 아니라 내가 편집기에서 입력합니다

NEO4J_PASSWORD 자리에 개인용 긴 비밀번호를 넣는다 — 기존 .env는 덮어쓰지 않는다

P0 P1 **P2** P3 P4 P5 P6 P7 P8 P9 P10 P11 P12 P13 P14

P2 · Claude Code 채팅에 붙여 넣기

```
.env.example과 docker-compose.yml을 읽고,
NEO4J_URI, NEO4J_USERNAME, NEO4J_DATABASE가
무엇인지 알려줘. .env가 없으면 .env.example을
복사하되, 비밀번호는 내 로컬 편집기에서 직접
설정하게 안내해줘. 비밀번호 값은 출력하거나
Git에 추가하지 마. 기존 Neo4j가 7474/7687을
쓰면 새 DB를 띄우지 말고 재사용 또는 포트
변경 방안을 먼저 설명해줘.
```

T2 · Windows

```
Copy-Item .env.example .env
notepad .env
```

T2 · macOS

```
cp .env.example .env
open -e .env
```

.env만 바꿔도 이미 만든 Docker 볼륨의 비밀번호는 바뀌지 않습니다

컨테이너는 기존 볼륨을 확인한 뒤에만 시작합니다

새 실습 DB가 필요할 때만 docker compose up -d — 결과는 Browser에서 RETURN 1로 확인한다

P3 · Claude Code 채팅에 붙여 넣기

```
기존 Neo4j 인스턴스와 데이터 볼륨을 먼저
확인해줘. 새 실습 DB가 필요한 경우에만
docker compose up -d를 실행하고, 컨테이너
상태와 로컬 포트 응답을 확인해줘.
Neo4j Browser(http://localhost:7474)에서
주소·사용자명·비밀번호를 어디서 읽는지만
알려줘. 비밀번호 값은 가려줘.
기존 볼륨은 절대 삭제하지 마.
```

T3 · 터미널

```
docker compose up -d
docker compose ps
```

C0 · Neo4j Browser

```
RETURN 1 AS ok;
```

성공 기준

실행 중 · 표에 1

Browser가 열려도 아직 기업 데이터는 없습니다 — 적재 성공이 아닙니다

03

데이터와 온톨로지 적재

P4 ~ P7 — CSV를 먼저 검증하고, 승인 후 MERGE로 Neo4j에 쓴다

DB에 넣기 전에 CSV 6개 검사부터 통과시킵니다

가상환경(.venv)을 만들거나 재사용하고 requirements-mcp.txt로 설치한다

P0 P1 P2 P3 **P4** P5 P6 P7 P8 P9 P10 P11 P12 P13 P14

P4 · Claude Code 채팅에 붙여 넣기

```
현재 프로젝트 폴더에서 Python 가상환경(.venv)을
만들거나 재사용해줘. requirements-mcp.txt를
사용해 필요한 패키지를 설치하고 설치 성공
여부를 확인해줘. 이어서
python -m unittest discover -s tests -v를
실행해 6개 데이터 무결성 검사의 실제
PASS/FAIL 목록을 보여줘. 실패하면 원인·
수정안·재실행 로그를 함께 보고해줘.
```

예상 출력

테스트 6건 ok
마지막 줄 Ran 6 tests ... OK

의미하지 않는 것

Neo4j 서버 연결은 아직 보증하지 않는다

CSV 단위 테스트 통과와 DB 적재 성공은 별개의 판정입니다

가상환경의 Python을 경로째 불러 실행합니다

T4 — 전역 python 대신 .venv 안의 Python을 직접 호출하면 ModuleNotFoundError가 줄어든다

P0 P1 P2 P3 **P4** P5 P6 P7 P8 P9 P10 P11 P12 P13 P14

T4 · Windows PowerShell

```
py -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements-mcp.txt
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

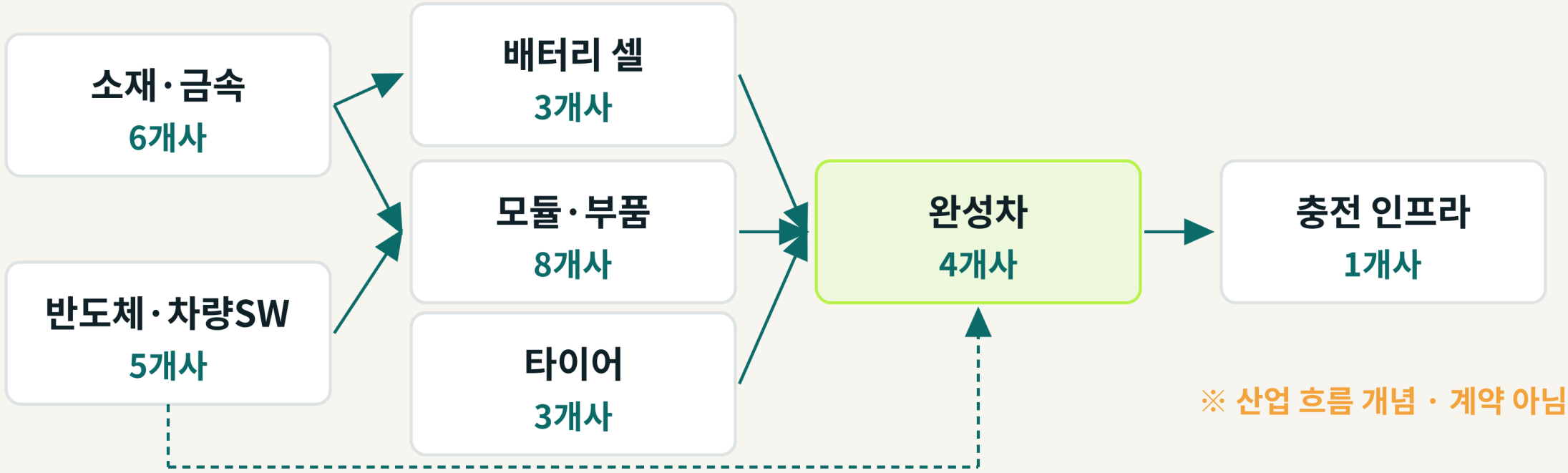
T4 · macOS / Linux

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements-mcp.txt
.venv/bin/python -m unittest discover -s tests -v
```

설치한 pip와 실행하는 Python이 같은 인터프리터인지가 첫 확인 대상입니다

7개 밸류체인은 완성차로 모이는 흐름입니다

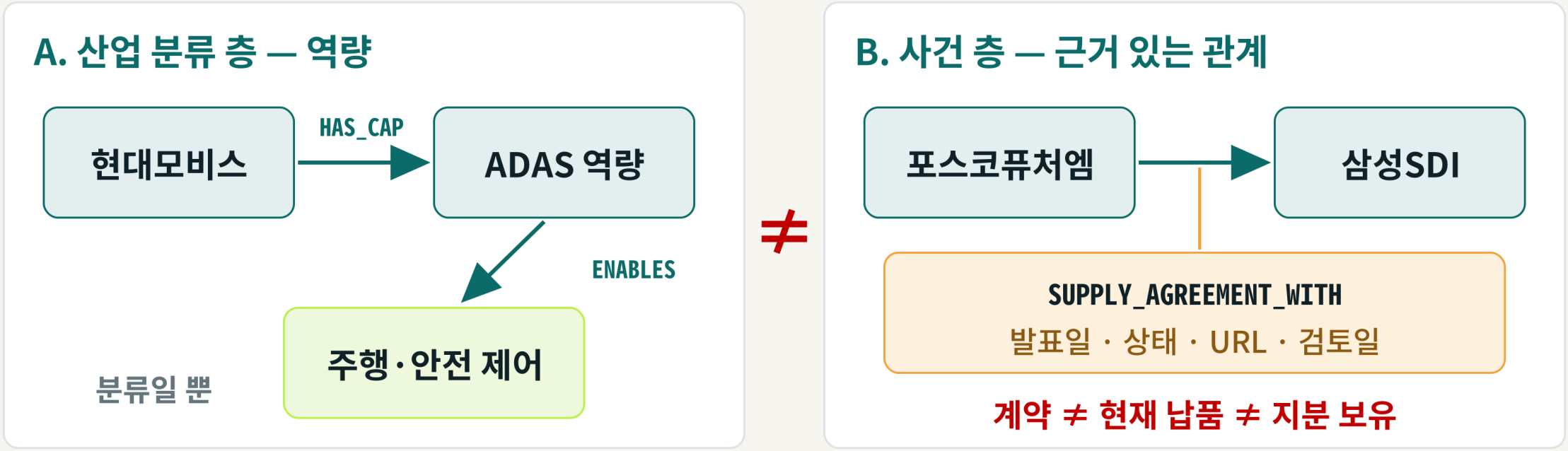
stage_links.csv의 TYPICALLY_FEEDS — 기업 30개를 대표 단계 하나씩에 배치한 학습용 분류



단계 사이의 화살표는 산업 흐름 개념이며, 한 회사가 여러 단계에 걸칠 수 있습니다

역량 연결과 기업 간 사건은 서로 다른 층입니다

A층: 기업 → 역량 → 차량 시스템(분류) · B층: 기업 ↔ 기업(근거 있는 사건)



현대모비스가 ADAS 역량에 연결돼도 어느 완성차에 공급한다는 증거는 아닙니다

노드 레이블 여섯 개가 CSV 파일과 짝을 이룹니다

레이블 이름 앞의 KAuto는 다른 프로젝트 데이터와 섞이지 않게 붙인 접두어다

KAutoCompany

기업 30

companies.csv

KAutoStage

밸류체인 7

stages.csv

KAutoCapability

핵심역량 24

capabilities.csv

KAutoSystem

차량 시스템 9

capabilities.csv

KAutoSource

공식 출처

evidence_url

KAutoRelationAssertion

관계 주장 42

relationships.csv

관계 타입: PARTICIPATES_IN_STAGE · HAS_CAPABILITY · ENABLES_SYSTEM + 기업 간 8종

스키마 설명을 요청할 때 사실과 분류를 다른 표로 받습니다

CSV와 import_neo4j.py를 읽게 하고, 모호한 관계는 새로 만들지 못하게 막는다

P5 · Claude Code 채팅에 붙여 넣기

```
이 프로젝트의 CSV와 scripts/import_neo4j.py를
읽고 한국 자동차 산업 온톨로지를 초보자에게
설명해줘. 노드 타입 6개 이상과 주요 관계
타입을 구분하고, 각각 실제 CSV 열에 매핑해줘.
기업→역량→차량 시스템과 기업↔기업 관계의
차이를 현대모비스, 포스코퓨처엠 사례로
비교해줘. 확인된 사실·산업분류·추정을 다른
표로 구분해줘. 모호한 관계는 만들지 마.
```

연습 질문 1

같은 그룹이면 직접 소유인가?
→ 아니다

연습 질문 2

EV 역량이 같으면 납품인가?
→ 아니다

Claude의 설명도 '확인된 사실'과 '분류'를 섞으면 다시 요청합니다

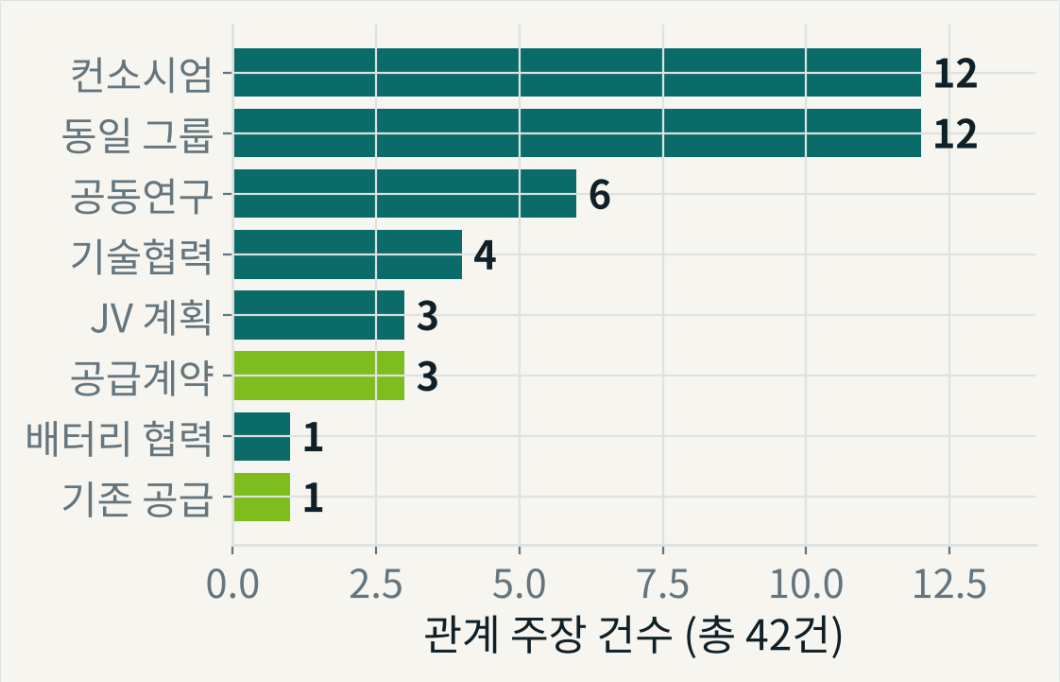
관계 42건 가운데 납품·계약은 4건뿐입니다

company_relationships.csv 실측 — 그룹·컨소시엄 관계가 24건으로 절반을 넘는다

읽는 법

선의 성격

- 그룹·컨소 24건
- 계약 3 · 공급 1
- 산업 비율 아님



선이 많은 기업이 거래가 많은 기업이라는 뜻은 아닙니다

관계 주장은 근거 · 시점 · 상태를 한 줄에 함께 저장합니다

RelationAssertion = 누가 · 누구와 · 관계 · 주제 · 발표일 · 상태 · URL · 요약 · 검토일

1
원자료
공시·IR·보도
`evidence_url`

2
사건
누가·누구와·무엇
`relation_type`

3
시점
발표일과 검토일 분리
`announcement_date`

4
상태
계약·계획·기존공급
`assertion_status`

5
검증
출처 수준 기록
`evidence_level`

판단 금지: 공급계약 발표 → 실제 납품 · 같은 그룹 → 지분 · 2홉 연결 → 위험 전파

열 하나라도 비면 그 관계는 '미확인'으로 남깁니다

발표일과 검토일은 서로 다른 시점을 뜻합니다

company_relationships.csv의 시간·근거 열 네 가지

열 이름	뜻	착각하기 쉬운 해석
announcement_date	발표·보도 당시 날짜	현재 사업 상태
reviewed_on	출처를 확인한 날 (2026-10-19)	계약 유효 확인일
assertion_status	signed · announced 등 시드 상태	실행 완료
evidence_url	실제 발표·보도 페이지	기업 대표 홈페이지

같은 A → B라도 공급계약과 기존 공급처럼 주장이 다르면 별도 ID로 둡니다

MERGE는 여러 번 실행해도 같은 관계를 한 번만 남깁니다

적재 스크립트가 중복을 막는 원리 — 재실행해도 42건이 84건이 되지 않는다

CREATE

매번 새로 만든다

- 실행할 때마다 추가
- 두 번 돌리면 중복
- 실습 적재에 쓰지 않음

42건 → 84건 위험

재실행

MERGE

있으면 찾고 없으면 만든다

- id 기준으로 일치 확인
- 재실행해도 1건 유지
- 기존 데이터 삭제 없음

import_neo4j.py 방식

MERGE도 쓰기 명령이므로 실행 전 승인 절차는 그대로 지킵니다

데이터 쓰기는 네 가지를 설명받고 승인한 뒤 실행합니다

운영 DB와 테스트 DB가 구별되면 테스트 DB를 쓴다

P6 · Claude Code 채팅에 붙여 넣기

```
데이터 적재 작업을 준비해줘.
(1) DB 주소/이름과 기존 데이터 존재 여부
(2) 새로 추가·수정할 노드·관계
(3) 백업 필요 여부 (4) MERGE 중복방지 로직을
먼저 설명해줘. 비밀번호나 연결 비밀을 출력하지
마. 실제 쓰기 작업 직전에 내 승인을 받고,
승인 후에만 scripts/import_neo4j.py를
실행해줘. 기존 데이터를 삭제하지 마.
```

T5 · macOS / Linux

```
.venv/bin/python
 scripts/import_neo4j.py
.venv/bin/python
 scripts/verify_neo4j.py
```

Windows

```
.venv\Scripts\python.exe
```

승인 전 설명 네 가지가 빠지면 '진행해'를 누르지 않습니다

검증 보고는 기대치와 실측치를 다른 열에 적게 합니다

verify_neo4j.py를 실제로 실행한 결과만 PASS — 못 돌렸으면 UNVERIFIED

P7 · Claude Code 채팅에 붙여 넣기

```
scripts/verify_neo4j.py를 실제로 실행하고
다음을 표로 보고해줘: 기업, 밸류체인 단계,
핵심역량, 차량시스템, 기업 간 관계,
관계 주장(Assertion), 출처 노드, 출처·상태를
가진 관계 수, 금지된 범용 SUPPLIES 관계 수.
기대치와 실제치를 별도 열로 두고 PASS/FAIL을
표시해줘. 실행하지 못했다면 UNVERIFIED로
남겨줘.
```

왜 SUPPLIES를 금지하나

'공급한다'는 뭉뚱그린 선은 계약과 납품을 섞는다

그래서

SUPPLIES_TO · SUPPLY_AGREEMENT_WITH로만 나눠 둔다

실행하지 못한 항목을 '완료'로 표시하지 않습니다

시드 기준값과 내 DB 실측값이 일치해야 통과입니다

교육용 데이터셋 기준값 — 한국 산업 전체의 기업·계약 총수가 아니다

검증 항목	기준값	내 실측값	판정
기업 (KAutoCompany)	30	____	PASS / FAIL
밸류체인 단계 · 핵심역량	7 · 24	____	PASS / FAIL
기업 간 관계 · 관계 주장	42 · 42	____	PASS / FAIL
범용 SUPPLIES 관계	0	____	PASS / FAIL

차량시스템 9 · 출처 12는 verify_neo4j.py의 실측값으로 확인합니다

Browser에서 개수와 출처를 직접 세어 봅니다

C1 · C2 — Neo4j Browser 상단 입력창에 붙여 넣는 읽기 전용 Cypher

C1 · 기업 간 관계를 타입별로 센다

```
MATCH (:KAutoCompany)-[r]->(:KAutoCompany)
WHERE r.project='kauto-stage1'
RETURN type(r) AS relation_type, count(*) AS relation_count ORDER BY relation_count DESC;
```

C2 · 관계와 출처를 함께 본다

```
MATCH (a:KAutoCompany)-[r]->(b:KAutoCompany)
WHERE r.project='kauto-stage1'
RETURN a.name_ko AS 출발기업, type(r) AS 관계, b.name_ko AS 도착기업,
       r.topic AS 주제, r.assertion_status AS 상태, r.evidence_url AS 근거URL
ORDER BY 관계, 출발기업 LIMIT 50;
```

WHERE r.project 조건이 이전 실습 데이터와 이번 시드를 가릅니다

04

그래프 탐색과 기업 간 관계 분석

P8 ~ P10 — 질문을 Cypher로 바꾸고, 선마다 출처를 연다

DB가 없어도 HTML 미리보기로 그래프를 먼저 봅니다

render_html.py가 CSV로 output/kauto_graph.html을 만든다 — Plan B의 출발점

P0 P1 P2 P3 P4 P5 P6 P7 **P8** P9 P10 P11 P12 P13 P14

P8 · Claude Code 채팅에 붙여 넣기

```
scripts/render_html.py를 실행해
output/kauto_graph.html을 만들고, 파일을
브라우저에서 여는 방법을 내 OS 기준으로
안내해줘. 기업을 선택하고 기업↔기업 관계유형
필터를 바꿀 때 어떤 노드와 선이 나타나는지
설명해줘. 반드시 이것이 CSV 기반 '정적
미리보기'이며 Neo4j 실시간 조회가 아니라는
점을 화면에 표시하고 보고해줘.
```

T6 · macOS

```
.venv/bin/python
 scripts/render_html.py
open output/kauto_graph.html
```

확인

기업 선택 · 관계유형 필터 · 출처 링크

강의 첫 5분에 완성 HTML을 보여 주고, 곧바로 만드는 명령 P8을 제시합니다

HTML은 클릭 경험이고 Browser는 실제 DB 질의입니다

둘의 선 개수가 같더라도 각각 따로 확인한다

output/kauto_graph.html

정적 미리보기

- CSV에서 직접 생성
- 클릭·필터·출처 링크
- DB가 꺼져도 열린다

적재 증거가 아니다



localhost:7474

Neo4j Browser

- 실제 DB에 Cypher 실행
- 표와 그래프로 결과
- count()로 적재 검증

적재 성공의 근거

Plan B 수강생은 HTML까지 성공하되, Neo4j·MCP 실습 완료로 표시하지 않습니다

자연어 질문 네 개를 읽기 전용 Cypher로 바꿉니다

질문마다 Cypher · 결과 형태 · 의미하지 않는 것 · 출처 확인법을 함께 받는다

P9 · Claude Code 채팅에 붙여 넣기

```
읽기 전용 Cypher만 사용해 한국 자동차 산업지도를 탐색하고 싶어.
다음 4개 질문에 대해 각각 (1) Cypher (2) 기대되는 그래프/표 형태
(3) 결과가 의미하지 않는 것 (4) 출처 확인 방법을 작성해줘.
A. 밸류체인별 기업 수          B. 현대모비스→핵심역량→차량 시스템
C. 현대자동차와 직접 연결된 기업·관계유형
D. 포스코퓨처엠 관련 공급계약과 실제 납품의 차이
프로젝트 queries/01, 02, 07, 08 파일을 먼저 읽고
필요하면 실제 스키마 이름에 맞춰 수정해줘.
```

'결과가 의미하지 않는 것'을 묻는 칸이 해석 오류를 미리 막습니다

질문 A와 B는 표와 그래프로 답이 나옵니다

C3는 표 보기, C4는 그래프 보기에서 실행한다 (queries/01 · 02)

C3 · A. 밸류체인별 기업 수 (표)

```
MATCH (c:KAutoCompany)-[:PARTICIPATES_IN_STAGE]->(s:KAutoStage)
RETURN s.name_ko AS 밸류체인, count(c) AS 기업수, collect(c.name_ko) AS 기업
ORDER BY 기업수 DESC;
```

C4 · B. 현대모비스 → 역량 → 차량 시스템 (그래프)

```
MATCH (c:KAutoCompany {id:'company.mobis'})-[r:HAS_CAPABILITY]->(p:KAutoCapability)
      -[e:ENABLES_SYSTEM]->(s:KAutoSystem)
RETURN c,r,p,e,s;
```

C4의 결과는 역량 분류이며, 현대모비스의 고객사 목록이 아닙니다

질문 D에서 공급계약과 실제 납품이 갈라집니다

C5 · C6 — 현대자동차 중심 네트워크와 공급 관계 두 타입의 건수 (queries/07 · 08)

C5 · C. 현대자동차와 직접 연결된 기업

```
MATCH p=(c:KAutoCompany {id:'company.hyundai'})
      -[r]-(other:KAutoCompany)
WHERE r.project='kauto-stage1' RETURN p LIMIT 50;
```

기존 공급 보도

SUPPLIES_TO

1건

C6 · D. 실제 납품 vs 공급계약

```
MATCH (:KAutoCompany)
      -[r:SUPPLIES_TO|SUPPLY_AGREEMENT_WITH]->(:KAutoCompany)
RETURN type(r) AS 관계유형, count(*) AS 수;
```

공급계약 발표

SUPPLY_AGREEMENT_WITH

3건

이 숫자는 시드의 출처 기록 수이며, 산업 전체의 납품·계약 수가 아닙니다

공식 발표로 확인한 관계만 선으로 그립니다

company_relationships.csv 가운데 다섯 건 — 오른쪽 날짜는 발표일(announcement_date)이다

포스코퓨처엠	LFP 양극재 공급계약 →	삼성SDI	2026-10-06
포스코퓨처엠	공급계약 + 기존 음극재 공급 →	SK온	2026-09-22
현대차·기아·모비스	북미 배터리셀 JV 계획 승인 →	SK온	2023-04-25
현대차·기아	배터리 공동연구 →	에코프로비엠	2024-09-25
현대차·기아	SDV 기술협력 →	삼성전자	2024-09-25

선은 관계 주장이며 물량 · 매출 · 지분율을 표현하지 않습니다

포스코퓨처엠과 SK온 사이에는 관계가 두 건 있습니다

2026-09-22 포스코퓨처엠 보도자료 하나에서 성격이 다른 주장 두 개를 분리했다 (rel_015 · rel_016)

rel_015 · SUPPLY_AGREEMENT_WITH

LFP 양극재 공급계약

- 2027~2029년 공급 예정
- 보도자료 제목: 약 1.1조 원
- 상태 contract_signed

계약 체결 ≠ 납품 개시

같은 A → B

rel_016 · SUPPLIES_TO

음극재 기존 공급

- 기존에 공급해 온 관계
- 같은 보도자료에 명시
- 상태 reported_existing_supply

현재 물량은 미확인

품목과 단계가 다르면 같은 두 회사라도 별도 ID로 저장합니다

JV 계획 승인은 현재 합작공장 운영을 뜻하지 않습니다

현대차·기아·현대모비스 → SK온 북미 배터리셀 합작투자 (rel_026~028, 2023-04-25)

원자료

2023년 이사회

합작사업 계획 승인

→

저장한 관계

JV 계획 관계

3건 (rel_026~028)

→

별도 검증

공장 가동 상태

직접 출자 구조

※ 현대차–LGES 배터리 협력(rel_029)도 그룹 협력의 대표 표시이며 JV 지분 소유가 아니다

발표 시점의 상태를 현재 상태로 옮겨 적지 않습니다

그래프에서 자주 나오는 비약 네 가지를 바로잡습니다

왼쪽 해석은 모두 근거가 없다 — 오른쪽의 추가 근거가 있어야 말할 수 있다

그래프에서 본 것	잘못된 해석	말하려면 필요한 근거
A와 B가 같은 그룹	A가 B 지분 100% 보유	지분 공시
A와 B가 협력 발표	A가 B에 정기 납품	공급 보도·계약
A → B → C 2홉 경로	A 차질 → C 매출 10% 감소	매출의존도·대체조달성
출처 URL이 있다	2026년 현재도 유효	계약 현재성 재조사

연결이 있다는 사실과 충격이 전파된다는 주장은 다른 층의 이야기입니다

관계 검증 프롬프트는 추정을 명시적으로 금지합니다

두 기업을 골라 공개자료 기반 관계를 여섯 항목으로 정리한다 — 근거가 없으면 '미확인'

P10 · Claude Code 채팅에 붙여 넣기

```
company_relationships.csv를 기준으로 현대자동차·현대모비스·포스코퓨처엠 중
두 기업을 골라 서로 연결된 공개자료 기반 관계를 탐색해줘.
각 결과를 (A) 관계 타입, (B) 방향, (C) 주제,
(D) 발표일, (E) 진척상태, (F) 1차 출처 URL로 정리해줘.
기업이 같은 그룹이라는 사실에서 소유비율을 추정하지 말고,
공급계약 발표에서 실제 납품량·매출액을 추정하지 마.
증거가 부족하면 '미확인'으로 표시해줘.
```

읽어볼 쿼리: queries/07_company_network · 08_supplier_network · 09_company_risk_paths

2홉 경로는 연결을 보여 줄 뿐 충격의 크기는 보여 주지 않습니다

C7 근거 공개 · C8 포스코퓨처엠 2홉 네트워크 (queries/09)

C7 · 연결별 근거 공개

```
MATCH (a:KAutoCompany)-[r]->(b:KAutoCompany)
WHERE r.project='kauto-stage1' AND r.evidence_url IS NOT NULL
RETURN a.name_ko AS A, type(r) AS 관계, b.name_ko AS B,
       r.announcement_date AS 발표일, r.assertion_status AS 상태, r.evidence_url AS 근거 LIMIT 20;
```

C8 · 2홉 기업 네트워크

```
MATCH p=(origin:KAutoCompany {id:'company.poscofuturem'})
  -[r1]-(mid:KAutoCompany)-[r2]-(dest:KAutoCompany)
WHERE origin<>dest AND all(r IN relationships(p)
  WHERE r.project='kauto-stage1') RETURN p LIMIT 30;
```

2단계에서 붙일 속성

매출의존도 · 지분비율
대체조달성

포스코퓨처엠의 차질이 2홉 너머 기업 매출에 자동으로 전파된다고 말할 수 없습니다

05

MCP 연결 · 개선 · 평가

P11 ~ P14 — 등록과 실제 호출을 분리해 검증하고, 결과를 보고서로 남긴다

MCP는 read-only로, 절대 경로로 한 번만 등록합니다

같은 이름의 MCP가 이미 있는지 먼저 확인하고, 비밀번호는 .env에서만 읽게 한다

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 P10 **P11** P12 P13 P14

P11 · Claude Code 채팅에 붙여 넣기

```
scripts/mcp_launcher.py, requirements-mcp.txt, README.md를 읽고
Neo4j MCP 서버를 현재 프로젝트에서 사용하도록 연결해줘.
먼저 같은 이름의 MCP가 등록되어 있는지 확인하고 중복 등록하지 마.
서버는 read-only여야 하고, 비밀번호는 .env에서만 읽어야 해.
현재 OS에 맞는 .venv Python과 mcp_launcher.py의 '절대 경로'로
claude mcp add 명령을 작성·실행하고 등록 결과를 확인해줘.
명령줄·설정 파일·로그에 비밀번호를 그대로 노출시키지 마.
```

권한 범위와 옵션은 현재 Claude Code 버전의 공식 문서로 확인합니다

등록 명령의 <...> 자리는 PC마다 다른 절대 경로가 들어갑니다

T7 — 그대로 복사해 실행하는 코드가 아니다. 실제 경로는 P11에게 만들게 한다

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 P10 **P11** P12 P13 P14

T7 · 형식

```
claude mcp add neo4j-kauto -- <가상환경_Python의_절대경로>
                              <scripts/mcp_launcher.py의_절대경로>
claude mcp list
```

등록 후 1 · 새 MCP 인식

세션 재시작

등록 후 2 · neo4j-kauto 확인

claude mcp list

등록 후 3 · 도구 목록

/mcp 상태 확인

명령줄 · 설정 파일 · 로그 어디에도 비밀번호가 찍히면 안 됩니다

MCP 질의는 사용한 도구명과 결과를 함께 보여 달라고 합니다

등록 목록에 보이는 것만으로는 성공이 아니다 — 실제 도구 호출 결과가 돌아와야 한다

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 P10 P11 **P12** P13 P14

P12 · Claude Code 채팅에 붙여 넣기

```
등록된 neo4j-kauto MCP 도구를 사용해서만
현재 DB를 읽어줘. (1) 노드 레이블 목록
(2) 기업 노드 개수 (3) 기업↔기업 관계
타입별 개수 (4) 현대모비스 연결 역량 5개
이내를 조회해줘. 사용한 도구명과 실행 결과를
보여줘. MCP 연결 실패 시 Python으로 대신
조회하고도 'MCP 성공'이라고 표시하지 마.
절대 데이터 쓰기 명령을 실행하지 마.
```

성공 판정

claude mcp list에 등록
AND 세션에서 실제 조회 결과

등록됨과 호출 성공은 다른 칸에 적습니다

성공 판정은 다섯 층으로 나눠 적습니다

아래층부터 차례로 — 위층이 성공해도 아래층 로그가 없으면 재현이 아니다

흔한 착각

한 층 건너뛰기

- 설치됨 ≠ 실행됨
- 화면 열림 ≠ 적재
- 등록 ≠ MCP 호출



각 층을 PASS · FAIL · UNVERIFIED 셋 중 하나로 판정합니다

자주 막히는 여섯 지점에는 진단 프롬프트가 준비돼 있습니다

D1~D6 — 공통 원칙: 원인을 구분해 확인 명령부터 하나씩, 기존 데이터는 건드리지 않는다

D1 · claude 없음

설치 vs PATH

OS·설치·PATH 구분

D2 · Docker 실패

CLI vs 엔진

Desktop 실행·WSL2 확인

D3 · 인증 실패

.env vs 볼륨

비밀번호 초기화 금지

D4 · ModuleNotFound

인터프리터 대조

.venv Python 직접 호출

D5 · MCP 호출 안 됨

경로·패키지·.env

/mcp에서 실제 질의

D6 · 42건이 아님

읽기로만 점검

DB명·중복 ID·project

어떤 진단에서도 전체 DB 초기화나 임의 삭제는 해법이 아닙니다

개선 요청은 설계와 테스트 계획을 먼저 받습니다

관계를 클릭하면 출처·상태가 보이도록 — 수정 전에 바뀌는 파일과 검증 방법부터 확인한다

P13 · Claude Code 채팅에 붙여 넣기

```
지금 생성한 기업 간 그래프의 어떤 관계든 클릭하거나 조회하면
(1) 연결 기업 (2) relation_type (3) 주제 (4) 발표일
(5) evidence_url (6) 현재상태 확인 여부를 볼 수 있게 개선안을 제시해줘.
HTML 화면과 Neo4j Cypher 중 어느 부분을 수정할지 먼저 설계하고
수정 전후 기능 목록과 테스트 계획을 보여줘.
출처 없는 신규 거래관계는 만들지 마.
파일 수정 전에 변경되는 파일과 검증방법을 알려줘.
```

설명가능성은 '선을 누르면 근거가 보인다'로 구현됩니다

새 관계 한 건은 공식 1차 출처부터 찾고 diff로 승인합니다

추가 순서: 출처 → 허용 타입 검사 → diff 승인 → 테스트 → HTML → MERGE → 읽기 조회

P0 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 | P11 | P12 | P13 | **P14**

P14 · Claude Code 채팅에 붙여 넣기

```
신규 기업 1곳 또는 기존 기업 사이 관계 1건을 추가하는 연습을 하고 싶어.
공식 1차 출처부터 찾고, 관계 타입이 허용 목록에 있는지 검사해줘.
발표일·진척상태·원문 핵심 문장·출처 URL을 별도 표로 정리하고,
CSV 수정 제안(diff)을 먼저 보여줘. 내가 승인하기 전에는 저장하지 마.
승인 후 데이터 단위 테스트, HTML 재생성, DB MERGE 적재와
read-only 조회를 순서대로 검증해줘.
증거 없는 항목은 추가하지 마.
```

증거 없는 항목은 그래프를 키우는 것이 아니라 오염시킵니다

여덟 칸짜리 틀 하나로 어떤 실습이든 다시 지시할 수 있습니다

재사용 가능한 프롬프트 틀 — 중괄호 { } 부분만 바꿔 쓴다

프롬프트 틀

```
[역할] 너는 {역할} 전문가이자 초보자 실습 코치다.
[목표] {이번 시간에 구현할 한 가지 결과}를 달성해줘.
[자료] 반드시 먼저 {파일명/폴더명}을 읽어줘.
[제약] {기존 데이터 삭제 금지, read-only, 증거 없는 관계 금지}.
[실행] ① 현황 점검 ② 계획 ③ 실행 ④ 테스트 ⑤ 최종 보고.
[승인] 실제 DB 쓰기/계정 설치/외부 공개 전에 내 승인을 받아.
[보고] 어디 → 입력 → 예상 출력 → 실제 출력 → 확인 기준.
[실패] 오류를 숨기지 말고 원인과 다음 검증 명령을 제시해줘.
```

[제약] · [승인] · [실패] 세 칸이 초보자의 사고를 막아 줍니다

한 줄 요청을 범위 · 출처 · 금지 · 검증이 있는 요청으로 고칩니다

같은 목표, 다른 프롬프트

부정확한 요청

“산업지도를 그려줘”

- 데이터 범위 불명확
- 출처 정의 없음
- 관계 정의 없음

그럴듯한 그림이 나온다

개선한 요청

기업 30 · 관계 42 기준

- 공식 출처 관계만 적재
- 테스트 6건 · 관계 42건 검증
- Cypher 3개 · 추정 납품 금지

검증 가능한 결과가 나온다

좋은 프롬프트는 결과물이 아니라 검증 방법을 미리 정합니다

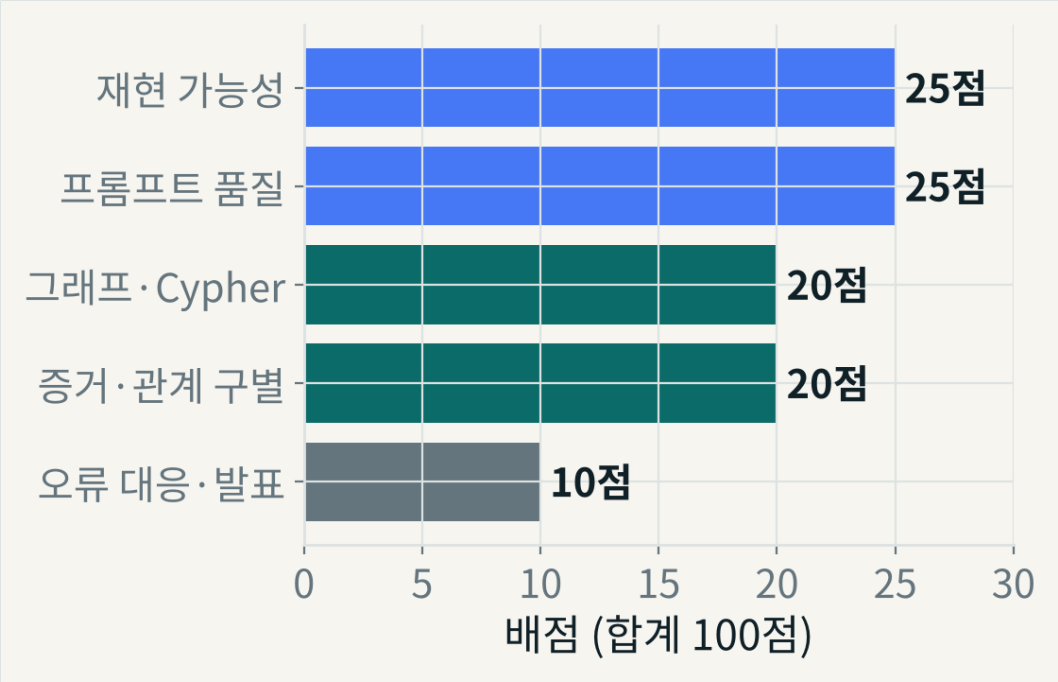
평가는 결과물보다 재현 과정에 더 많이 배점합니다

권장 평가표 100점 — 재현 가능성과 프롬프트 품질이 절반을 차지한다

보너스

오류를 푼 기록

- 실제 오류를 만남
- 체계적으로 해결
- 왜 성공했는지 증명



‘실패가 없었다’보다 ‘왜 성공했는지 증명할 수 있다’가 중요합니다

심화 과제 세 개는 모두 해석의 정확성을 묻습니다

제출은 student_report.md 한 파일 — 템플릿은 Vault의 student_report_TEMPLATE.md

과제 A

잘못된 설명 고치기

- "배터리사와 연결 = 전량 납품"
- 증거·계약·이행 구분
- 5줄 이내 수정

과제 B

2홉 경로의 의미

- queries/09 실행
- 경로 하나의 타입·증거
- 암시하지 않는 것 보고

과제 C

내 질문 만들기

- 같은 시스템, 납품 미검증?
- Cypher 초안 → 오류 수정
- 전 과정 기록

세 과제 모두 '그래프가 말하지 않는 것'을 적어야 만점입니다

Docker가 막힌 교실에서는 Plan B로 먼저 성공시킵니다

수업 전 점검: 관리자 권한 · 네트워크 · 포트 7474/7687 · Claude 로그인 · Python 실행

PLAN A · 설치 가능

전체 실습

- Docker + Neo4j + MCP
- 실제 적재·Browser·MCP
- 수치 수정·비교까지

다섯 층 모두 판정

PLAN B · 설치 제약

데이터 중심 실습

- CSV 테스트 + 정적 HTML
- Cypher 읽기·설계 연습
- DB는 강사 시연·공유 DB

Neo4j·MCP는 미완료로 표시

서로 다른 OS의 설치 화면을 동시에 따라 하게 하지 않습니다

1단계의 출처 구조는 2단계 금융 온톨로지에서 그대로 쓰입니다

이번 실습 범위는 1단계까지 — NeoQuant · GraphRAG는 확장 설계만 소개한다

1단계 · 자동차 산업지도

✓ 기업 ↔ 기업 계약·협력

✓ 기업 → 소재·역량·시스템

✓ 공식 근거 · 시점 · 상태

✓ CSV → Neo4j → MCP 검증

재사용

Source·Assertion

2단계 · NeoQuant / GraphRAG

+ 증권 ↔ 팩터 연결

+ 기업 ↔ 신용·리스크 노출

+ 논문·전략·데이터 근거 추적

+ 벡터 검색 + 그래프 탐색

그래프 경로는 가설 생성과 증거 추적의 도구이며, 손실 추정의 완성품이 아닙니다

필요한 명령은 이 한 장에 모두 있습니다

macOS/Linux 기준 — Windows는 .venv\Scripts\python.exe · Get-Location으로 바꾼다

명령어 한 페이지

```
pwd                                                  # 폴더 위치 확인
claude                                               # Claude Code 시작
docker compose ps / up -d                            # Docker 확인 / DB 시작
python3 -m venv .venv                                # 가상환경 생성
.venv/bin/python -m pip install -r requirements-mcp.txt  # 패키지 설치
.venv/bin/python -m unittest discover -s tests -v    # 단위 테스트
.venv/bin/python scripts/render_html.py              # HTML 생성
.venv/bin/python scripts/import_neo4j.py             # DB 적재(승인 후)
.venv/bin/python scripts/verify_neo4j.py             # DB 검증
claude mcp list                                      # MCP 등록 확인
```

Neo4j Browser는 localhost:7474 — 명령 앞에 항상 '지금 어느 폴더인가'를 먼저 확인합니다

수업 전체를 한 번에 진행하는 오케스트레이션 프롬프트입니다

메인 프롬프트 · 모든 실습을 순서대로

```
프로젝트의 README.md, CLAUDE_SETUP_PROMPT.md, CLAUDE.md, RELATIONSHIP_MODEL_KR.md,
EXERCISES_KR.md를 먼저 읽어줘. 나는 초보 수강생이다. K-Auto KG 실습을 이 PC에서 재현하고 싶다.
단계를 (1) 환경진단 (2) 비밀정보 설정 (3) Neo4j 실행 (4) Python 설치·CSV 테스트
(5) 정적 HTML (6) 온톨로지 설명 (7) 사용자 승인 후 DB 적재 (8) Cypher 조회
(9) read-only MCP 연결 (10) 완료 체크리스트로 나눠줘.
각 단계에서 '어디 → 입력 → 실제 출력 → PASS/FAIL → 다음 단계' 순서로 보고하고,
사용자가 직접 해야 하는 일은 한 번에 하나씩 안내해줘.
비밀번호는 출력하지 말고, 기존 DB·볼륨·파일 삭제는 금지해.
기업과 기업의 관계는 근거가 있는 42건의 시드만 사용하고,
실제 납품과 공급계약, 계열과 소유관계를 반드시 구분해.
실행하지 못한 것은 완료라고 표시하지 마. 완료 후 student_report.md 초안을 만들어줘.
```

P0~P14를 익힌 뒤에 쓰는 요약판이며, 첫 수업에서는 단계별 프롬프트를 씁니다

수업을 마치기 전 여덟 칸을 실측값으로 채웁니다

수업 종료 체크리스트 — 실행 환경이 허용하지 않은 칸은 UNVERIFIED로 남긴다

데이터 · DB

실측 확인

- 단위 테스트 6개 PASS
- HTML 열기·필터 체험
- 30 · 7 · 24 · 42 실측
- Cypher 3종 실행

MCP · 보고

증명 확인

- MCP 실제 읽기 질의
- 관계 1건 출처·상태
- student_report.md 작성
- 비밀정보 삭제 확인

프롬프트 · 로그 · 구별 · 출처 · Cypher · MCP 결과를 설명할 수 있으면 통과입니다

입력한 명령을 남기고

선마다 출처를 열고

실행한 것만 완료로 적는다

그래프는 가설을 만들고,
근거가 그 가설을 지킵니다.

※ 공개자료 기반 교육용 실습 교재 — 기업 간 경제적 관계를 확정하거나 투자 판단의 유일한 근거로 쓰지 않는다. 출처 검토일 2026-10-19.